

Turnos

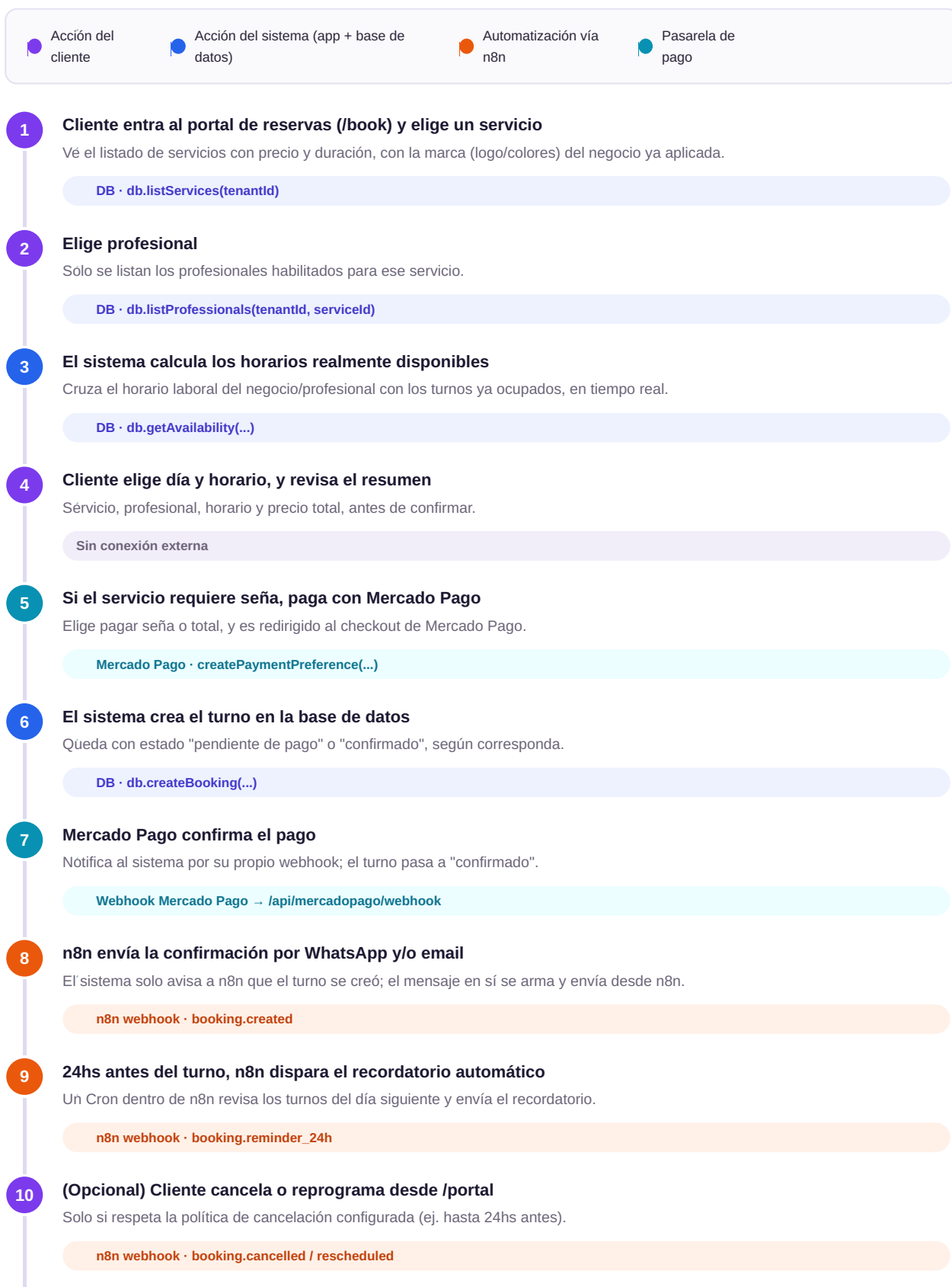
Flujo de usuario, integraciones y Git

Cómo se mueve un turno de punta a punta, qué conexiones hay que activar en cada paso, y cómo empezar a versionar el proyecto en un repositorio.

Complementa a: Turnos-Guia-de-inicio.pdf y turnos-saas.zip
Para: equipo técnico que va a evaluar / continuar el desarrollo

1 Flujo de usuario modelo

Recorrido completo de un turno reservado por un cliente, desde que entra al portal hasta que es atendido. Cada paso indica quién actúa y, si corresponde, qué integración se dispara.



11

Día del turno: el empleado marca asistencia y el dueño ve todo en el dashboard

El empleado actualiza el estado desde su agenda; el turno queda en el historial del cliente (CRM).

DB · `db.updateBooking(...)`

Para el equipo que analiza esto: los pasos 1, 2, 3, 6 y 11 ya están resueltos en el código (con datos de ejemplo). Los pasos 5, 7, 8 y 9 son los que requieren tener las cuentas de Mercado Pago y n8n activas para poder probarlos de punta a punta.

2 Puntos de conexión, automatización y credenciales

Desglose de cada punto del flujo anterior que depende de un servicio externo: qué credencial hay que conseguir y en qué variable de entorno se carga (todas viven en `.env.local`, plantilla en `.env.example`).

PUNTO DEL FLUJO	SERVICIO	CREDENCIAL NECESARIA	VARIABLE DE ENTORNO
Catálogo de servicios, profesionales, disponibilidad y CRM	Firebase o Airtable	Firebase: apiKey, authDomain, projectId, etc. · Airtable: API key + Base ID	<code>NEXT_PUBLIC_FIREBASE_*</code> o <code>AIRTABLE_API_KEY</code> / <code>AIRTABLE_BASE_ID</code>
Login de dueño, empleado y cliente	Firebase Authentication	Mismo proyecto Firebase de arriba, con el método de acceso activado (Email/Password)	<code>NEXT_PUBLIC_FIREBASE_*</code>
Escrituras privilegiadas desde el servidor (opcional, más seguro)	Firebase Admin SDK	Cuenta de servicio (Service Account) generada desde Firebase Console	<code>FIREBASE_ADMIN_PROJECT_ID</code> , <code>FIREBASE_ADMIN_CLIENT_EMAIL</code> , <code>FIREBASE_ADMIN_PRIVATE_KEY</code>
Cobro de seña o total al confirmar el turno	Mercado Pago (Checkout)	Access Token + Public Key de prueba (sandbox)	<code>MERCADOPAGO_ACCESS_TOKEN</code> , <code>NEXT_PUBLIC_MERCADOPAGO_PUBLIC_KEY</code>
Confirmación automática del pago	Webhook de Mercado Pago	URL pública de la app (ver nota abajo) + secreto propio para validar el webhook	<code>MERCADOPAGO_WEBHOOK_SECRET</code> , <code>NEXT_PUBLIC_APP_URL</code>
Aviso de turno reservado / cancelado / reprogramado	n8n (Webhook nodes)	URL de cada Webhook creado en n8n + secreto compartido	<code>N8N_WEBHOOK_BOOKING_CREATED</code> , <code>_CANCELLED</code> , <code>_RESCHEDULED</code> , <code>N8N_WEBHOOK_SECRET</code>
Recordatorio 24hs antes del turno	n8n (nodo Cron + Webhook)	Igual que arriba, más el nodo Cron configurado adentro de n8n (no requiere credencial extra)	<code>N8N_WEBHOOK_REMINDER_24H</code>
Envío real del WhatsApp (dentro del workflow de n8n)	WhatsApp Business API o Twilio	Cuenta y credenciales del proveedor de WhatsApp elegido — no está cubierto por este MVP , se configura directo en n8n	Se guarda como credencial dentro de n8n, no en este proyecto
Envío real del email (dentro del workflow de n8n)	SMTP / SendGrid / Gmail API	Credenciales del servicio de email elegido	Se guarda como credencial dentro de n8n, no en este proyecto

A tener en cuenta: para que Mercado Pago y n8n puedan "avisarle" al sistema (webhooks entrantes), la app tiene que tener una URL pública. En desarrollo local eso se resuelve con un túnel temporal como `ngrok`, o directamente probando esta parte una vez que el proyecto esté desplegado (ej. en Vercel).

3 Primeros pasos en consola: subir esto a un repositorio

Guía ágil para practicar Git desde cero con este mismo proyecto: desde instalarlo hasta tener el código en GitHub.

Paso 0 — Verificar que Git está instalado

```
git --version
# si no devuelve nada, instalar desde https://git-scm.com/downloads
```

Paso 1 — Configurar tu identidad (una sola vez por computadora)

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-email@ejemplo.com"
```

Paso 2 — Entrar a la carpeta del proyecto

```
cd turnos-saas
```

Paso 3 — Convertir la carpeta en un repositorio Git

```
git init
git add .
git commit -m "Primer commit: esqueleto MVP de Turnos"
```

El proyecto ya incluye un archivo `.gitignore` que excluye `node_modules`, `.env.local` y archivos temporales, así que ese `git add .` no va a subir credenciales ni carpetas pesadas por error.

Paso 4 — Crear el repositorio vacío en GitHub

#	ACCIÓN
1	Entrar a https://github.com/new (crear cuenta gratis si no se tiene).
2	Nombre del repositorio: por ejemplo <code>turnos-saas</code> .
3	Elegir Private (recomendado, mientras se evalúa el proyecto).
4	No tildar "Add a README" (el proyecto ya trae uno) y crear.
5	GitHub va a mostrar una URL como https://github.com/tu-usuario/turnos-saas.git — copiarla.

Paso 5 — Conectar el repositorio local con GitHub y subir el código

```
git remote add origin https://github.com/tu-usuario/turnos-saas.git
git branch -M main
git push -u origin main
```

La primera vez que se hace `push`, GitHub va a pedir iniciar sesión (usuario + token de acceso personal, no la contraseña de la cuenta). Se genera en `github.com` → `Settings` → `Developer settings` → `Personal access tokens`.

Rutina diaria de trabajo (a partir de acá)

```
git status          # qué archivos cambiaron
git add .           # preparar los cambios
git commit -m "Descripción breve del cambio"
git push            # subir a GitHub
git pull            # traer cambios de otros compañeros
```

Chuleta de comandos esenciales

Para empezar a trabajar

```
git clone URL — bajar un repo existente
git status — ver qué cambió
git log --oneline — ver el historial
```

Para guardar cambios

```
git add archivo.ts — sumar un archivo puntual
git add . — sumar todos los cambios
git commit -m "mensaje" — guardar el punto
```

Para trabajar en paralelo

```
git branch — ver las ramas
git checkout -b feature/nombre — crear y moverse a
una rama nueva
git checkout main — volver a la rama principal
```

Para sincronizar con GitHub

```
git push — subir commits
git pull — bajar commits nuevos
git remote -v — ver a qué repo está conectado
```

Tip para practicar sin miedo: cualquier error de Git dentro de una rama nueva (`feature/...`) es fácil de descartar — alcanza con volver a `main` y borrar la rama. Es un buen espacio para practicar antes de tocar el código "real" del proyecto.

Este documento complementa a [Turnos-Guia-de-inicio.pdf](#) y al proyecto en [turnos-saas.zip](#). El detalle técnico ampliado de cada servicio sigue estando en el [README.md](#) incluido en el ZIP.